

Búsqueda adversaria

Minimax y poda alfa-beta

Joaquín F. Sánchez

Curso de Inteligencia Artificial

2026

Cuando existe un oponente, el entorno también decide

Búsqueda de un agente

El agente elige acciones en un entorno que no intenta perjudicar su objetivo.

Búsqueda adversaria

Cada decisión cambia las opciones del rival, quien también elige racionalmente.

No basta con elegir una buena jugada: hay que anticipar la mejor respuesta.

Al finalizar la sesión, el estudiante podrá:

- formular un juego como un problema de búsqueda adversaria;
- construir y evaluar un árbol de juego;
- ejecutar Minimax mediante propagación de utilidades;
- aplicar los límites α y β para realizar podas;
- comparar el costo computacional de Minimax y alfa-beta.

Un juego define estados, acciones y utilidad

Un problema adversario determinista de dos jugadores se modela con:

- **Estado inicial:** configuración antes de decidir.
- **Jugador:** indica quién mueve en cada estado.
- **Acciones:** jugadas legales disponibles.
- **Resultado:** estado producido por una acción.
- **Prueba terminal:** victoria, derrota, empate o fin del juego.
- **Utilidad:** valor numérico del resultado para MAX.

MAX y MIN representan intereses opuestos

Jugador MAX

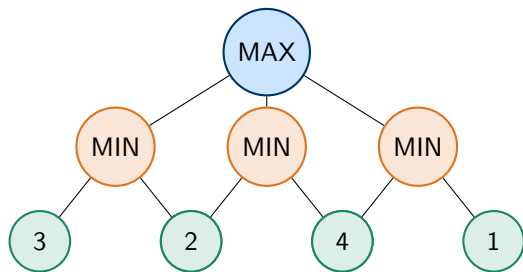
Selecciona la acción con el valor más alto.
Representa al agente que toma la decisión.

Jugador MIN

Selecciona la acción con el valor más bajo.
Representa al oponente racional.

$$U(\text{victoria}) = +1, \quad U(\text{empate}) = 0, \quad U(\text{derrota}) = -1$$

El árbol alterna decisiones de ambos jugadores



Cada nivel representa una jugada; las hojas contienen utilidades o evaluaciones.

Minimax supone que ambos jugadores actúan racionalmente

Principio

MAX escoge la acción cuyo peor resultado posible sea el mejor entre sus alternativas.

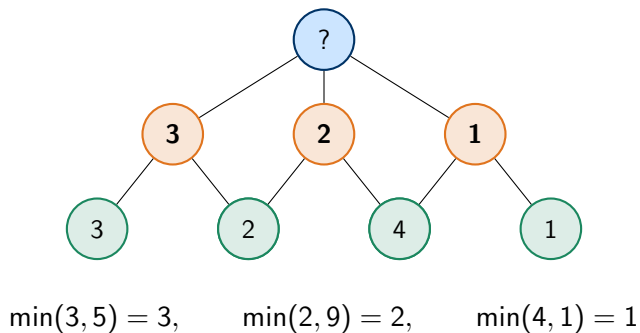
- MAX intenta aumentar la utilidad.
- MIN intenta reducir la utilidad de MAX.
- No se supone que el rival cometerá un error evitable.
- La decisión es robusta frente a la mejor respuesta del oponente.

Los valores se propagan desde las hojas hasta la raíz

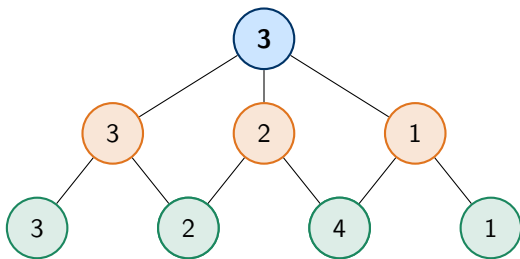
$$V(s) = \begin{cases} U(s), & \text{si } s \text{ es terminal,} \\ \max_{a \in A(s)} V(\text{Resultado}(s, a)), & \text{si juega MAX,} \\ \min_{a \in A(s)} V(\text{Resultado}(s, a)), & \text{si juega MIN.} \end{cases}$$

La raíz elige la acción asociada al mayor valor Minimax.

Paso 1: MIN conserva el menor valor de cada rama



Paso 2: MAX elige la mejor de las peores respuestas



$$\max(3, 2, 1) = 3$$

MAX elige la primera acción porque garantiza una utilidad de al menos 3.

Pseudocódigo recursivo de Minimax

```
MINIMAX(estado, profundidad, es_max):  
    si TERMINAL(estado) o profundidad = 0:  
        retornar EVALUAR(estado)  
  
    si es_max:  
        valor = -infinito  
        para hijo en SUCESTORES(estado):  
            valor = max(valor, MINIMAX(hijo, profundidad-1, falso))  
    de lo contrario:  
        valor = +infinito  
        para hijo en SUCESTORES(estado):  
            valor = min(valor, MINIMAX(hijo, profundidad-1, verdadero))  
    retornar valor
```

La profundidad limitada exige una función de evaluación

Explorar hasta todos los estados terminales suele ser inviable. Al alcanzar el límite:

$$V(s) \approx \text{Eval}(s)$$

Una buena evaluación debe:

- asignar valores altos a estados favorables para MAX;
- asignar valores bajos a estados favorables para MIN;
- distinguir estados no terminales relevantes;
- calcularse mucho más rápido que continuar la búsqueda.

Ejemplo de evaluación para Conecta Cuatro

Una función sencilla puede combinar características del tablero:

$$\text{Eval}(s) = 100L_4 + 10L_3 + 2L_2 - (100R_4 + 10R_3 + 2R_2)$$

- L_k : alineaciones potenciales de longitud k para MAX.
- R_k : alineaciones potenciales de longitud k para MIN.
- Los pesos expresan qué patrones son más importantes.

La evaluación orienta la decisión, pero no es una garantía fuera del horizonte.

El árbol crece exponencialmente

Si cada estado tiene en promedio b acciones y se exploran m niveles:

Tiempo = $O(b^m)$, Memoria = $O(bm)$ con recorrido en profundidad

- Aumentar un nivel multiplica aproximadamente el trabajo por b .
- En juegos reales no suele ser posible recorrer todo el árbol.
- Se necesitan límites de profundidad y técnicas de poda.

Alfa-beta evita explorar ramas que no pueden cambiar la decisión

La poda alfa-beta produce **el mismo valor y la misma jugada** que Minimax completo, pero omite subárboles irrelevantes.

Idea

Si una rama ya es peor que una alternativa conocida, un jugador racional nunca la elegirá.

No modifica la función de utilidad: modifica cuánto árbol se examina.

α y β resumen las mejores garantías conocidas

- α : mejor valor que MAX puede garantizar hasta el momento.
- β : mejor valor que MIN puede garantizar hasta el momento.

$$\alpha = -\infty, \quad \beta = +\infty$$

Durante el recorrido:

$$\alpha \leftarrow \max(\alpha, v) \quad \text{en MAX}, \quad \beta \leftarrow \min(\beta, v) \quad \text{en MIN}$$

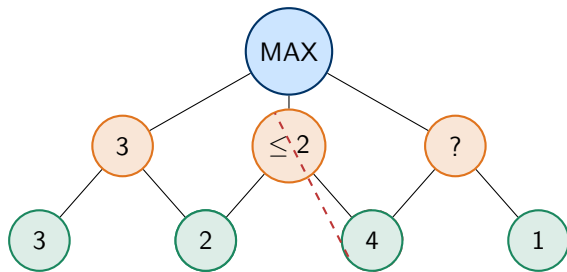
La condición de corte es $\alpha \geq \beta$

Corte alfa-beta

Cuando $\alpha \geq \beta$, los hijos restantes del nodo actual no pueden afectar la elección de un antecesor racional.

- En un nodo MIN, se poda cuando su valor ya es $\leq \alpha$.
- En un nodo MAX, se poda cuando su valor ya es $\geq \beta$.
- Las ramas podadas no son necesariamente malas en términos absolutos; son irrelevantes para la decisión actual.

Ejemplo: una respuesta de MIN permite podar



La primera rama fija $\alpha = 3$. En la segunda, MIN encuentra 2: como $2 \leq \alpha$, sus hijos restantes no pueden mejorar la decisión de MAX.

Pseudocódigo de alfa-beta

```
ALFA_BETA(estado, profundidad, alfa, beta, es_max):  
    si TERMINAL(estado) o profundidad = 0: retornar EVALUAR(estado)  
    si es_max:  
        valor = -infinito  
        para hijo en SUCESORES(estado):  
            valor = max(valor, ALFA_BETA(hijo, profundidad-1,  
                                          alfa, beta, falso))  
            alfa = max(alfa, valor)  
            si alfa >= beta: romper  
    de lo contrario:  
        valor = +infinito  
        para hijo en SUCESORES(estado):  
            valor = min(valor, ALFA_BETA(hijo, profundidad-1,  
                                          alfa, beta, verdadero))  
            beta = min(beta, valor)  
            si alfa >= beta: romper  
    retornar valor
```

El orden de las jugadas determina cuánto poda se logra

- Si se examinan primero las mejores jugadas, α y β se ajustan pronto.
- En el peor orden, alfa-beta se aproxima a $O(b^m)$.
- Con orden ideal, puede aproximarse a $O(b^{m/2})$.
- Esto permite explorar aproximadamente el doble de profundidad con recursos comparables.

Heurística práctica

Ordenar primero capturas, amenazas, jugadas ganadoras o movimientos con alta evaluación.

Minimax y alfa-beta toman la misma decisión

	Minimax	Alfa-beta
Valor final	Óptimo frente a rival racional	El mismo valor
Nodos	Examina todo el árbol limitado	Omite ramas irrelevantes
Peor caso	$O(b^m)$	$O(b^m)$
Orden ideal	$O(b^m)$	Aproximadamente $O(b^{m/2})$
Resultado	Mejor acción	La misma mejor acción

Errores frecuentes al implementar búsqueda adversaria

- Evaluar desde la perspectiva del jugador equivocado.
- No alternar correctamente MAX y MIN.
- Modificar el mismo estado sin copiarlo antes de simular una acción.
- Omitir las condiciones terminales o el límite de profundidad.
- Actualizar α en MIN o β en MAX de forma incorrecta.
- Suponer que la poda cambia el resultado de Minimax.

Actividad guiada: propagar y podar

Para un árbol de tres niveles:

- 1 identifique cuáles niveles corresponden a MAX y MIN;
- 2 asigne utilidades a las hojas;
- 3 propague los valores aplicando Minimax;
- 4 repita de izquierda a derecha registrando α y β ;
- 5 marque las ramas que no necesitan evaluarse;
- 6 verifique que ambos métodos eligen la misma acción.

Un nodo MAX tiene tres hijos MIN con utilidades terminales:

$$A = (5, 6), \quad B = (4, 9), \quad C = (7, 2)$$

- 1 ¿Qué valor propaga cada nodo MIN?
- 2 ¿Qué acción selecciona MAX?
- 3 Recorriendo de izquierda a derecha, ¿qué hojas puede podar alfa-beta?

Un nodo MAX tiene tres hijos MIN con utilidades terminales:

$$A = (5, 6), \quad B = (4, 9), \quad C = (7, 2)$$

- 1 ¿Qué valor propaga cada nodo MIN?
- 2 ¿Qué acción selecciona MAX?
- 3 Recorriendo de izquierda a derecha, ¿qué hojas puede podar alfa-beta?

Solución: $A = 5$, $B = 4$, $C = 2$; MAX elige A. En B se poda la hoja 9 porque $\beta = 4 \leq \alpha = 5$. En C deben evaluarse ambas hojas.

Síntesis: anticipar, evaluar y descartar

- Minimax alterna decisiones de MAX y MIN.
- Las utilidades se propagan desde las hojas hasta la raíz.
- La profundidad limitada requiere una evaluación heurística.
- Alfa-beta descarta ramas incapaces de cambiar la decisión.
- El orden de exploración mejora la eficacia de la poda.

Buscar bien es anticipar la mejor respuesta posible.

-  Material suministrado para la sesión. *Intelligent Search*, capítulo 3, pp. 72–89.
-  Russell, S. y Norvig, P. *Artificial Intelligence: A Modern Approach*. Pearson.
-  Knuth, D. y Moore, R. “An Analysis of Alpha-Beta Pruning”. *Artificial Intelligence*, 1975.